Single Trajectory Based Metaheuristics II

Heuristic Optimization Techniques 192.162, VU 6 ECTS

Enrico Iurlano, Günther Raidl, and Laurenz Tomandl
heuopt@ac.tuwien.ac.at

Algorithms and Complexity Group
Institute of Logic and Computation, TU Wien

WS 2026

Version from October 29, 2026



Simulated Annealing

Simulated annealing (SA)

Kirkpatrick et al. (1983)

- First metaheuristic building on top of local search to escape local optima
- Inspired by the physical process of annealing to obtain a strong crystalline structure
  $\rightarrow$ nature inspired method
- Basic idea:
  Even worse solutions are accepted with a certain probability
- Usually, SA is based on a random neighbor step function

procedure simulated annealing

```
begin
  t ← 0;
  T ← T_init;
  x ← initial solution;
  x* ← x;
  repeat:
    repeat:
      choose an x' ∈ N(x) randomly;
      if x' is better than x then
        x ← x';
      else
        if P < e^{-|f(x')-f(x)|/T} then
          x ← x';
      if x is better than x* then x* ← x;
      t ← t + 1;
    until equilibrium condition satisfied;
    T ← g(T, t);      // cooling
  until stopping criteria satisfied;
  return x*;
end
```

Metropolis criterion:

$$P < e^{\frac{-|f(x')-f(x)|}{T}}$$

$P \ldots$ random number $\in [0, 1)$

Annealing:

Temperature T is slowly decreased

Parameters for the cooling schedule

- Initial temperature T_{init}
  - e.g., based on known bounds: $T_{\text{init}} = f_{\text{max}} - f_{\text{min}}$
  - e.g., so that $\approx 3\%$ of the moves are rejected at the beginning
- Cooling schedule $g(T, t)$
  - Geometric cooling:
    $g(T, t) = T \cdot \alpha$, $0 < \alpha < 1$, (e.g., $\alpha = 0.95$)
- Equilibrium condition:
  - Static: Having accomplished a fixed number of iterations, e.g., multiple of the neighborhood size
  - Adaptive: dynamic number of moves, e.g., a given number of moves without improvement
- Stopping criteria
  - e.g., reaching a final temperature
  - or no improvement over τ levels of temperature

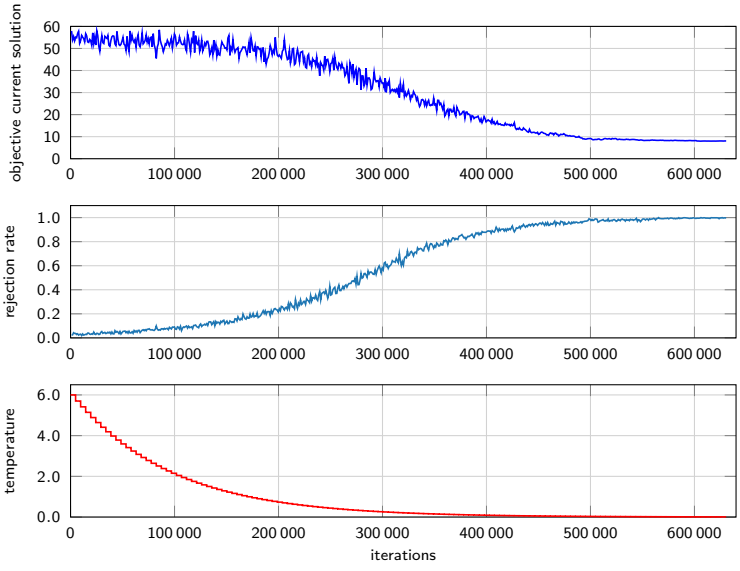
Example: Simulated annealing for the TSP

- Simple implementation: Johnson and McGeoch (1997)
  - Start with a random tour
  - 2-exchange neighborhood
  - Cooling schedule:
    - T_{init} so that $\approx 3\%$ of moves are rejected
    - Geometric cooling, $\alpha = 0.95$
    - Iterations to satisfy equilibrium: $n \cdot (n-1)$
    - Stopping criteria: 5 temperature levels without improvement

$\rightarrow$ much better results than pure 2-opt LS with longer running times
$\rightarrow$ relatively poor results compared to LS with 3-opt moves or Lin-Kernighan

Example: Simulated annealing for the TSP (cont.)

Exemplary run



Example: Simulated annealing for the TSP (cont.)

- Significant improvements
  - Restriction of the neighborhood to promising moves, e.g., neighborhood pruning on geometric instances by inscribing the instance into a large enough square being subdivided equidistantly into m^2 subcells ($m \in \mathbb{N}$); edges for the 2-opt move are only chosen among the edges lying in the same/adjacent subcell (Bonomi and Lutton, 1984)
- Nevertheless: For the TSP, SA is not competitive in particular when compared to leading TSP methods.

$\rightsquigarrow$ Now, an interactive demo on this ...

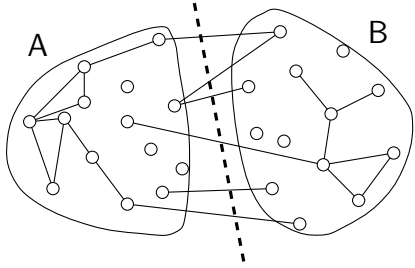
Example: SA for graph bipartitioning

Definition (GRAPH BIPARTITIONING, see Johnson et al. (1989))

Given: graph $G = (V, E)$, $n := |V|$, n even

To be searched: partition of V consisting of two vertex sets V_1, V_2, with $|V_1| = |V_2|$, $V_1 \cap V_2 = \emptyset$, and $V_1 \cup V_2 = V$,

minimizing $f(V_1, V_2) := |\{\{u, v\} \in E \mid u \in V_1 \wedge v \in V_2\}|$
(i.e., "minimizing the cutsize")



Example: SA for graph bipartitioning (cont.)

- Solution representation:

  Characteristic vector $x = (x_1, \ldots, x_n)$, $x_i \in \{1, 2\}$
  $\rightarrow$ node i is assigned to set V_{x_i}

- Simulated annealing (Johnson et al., 1989)
  - 2-node-exchange neighborhood
  - Random initial solution
  - Similar cooling schedule as for the TSP
  - One of the first applications of SA

Example: SA for graph bipartitioning (cont.)

- Improvements:
  - **Permit infeasible solutions → but use smaller neighborhood**
  - *New neighborhood:*

    Move single node into the other set
    - $|N(x)| = n$ instead of $n^2/4$
  - *Modified objective function:*

    $f(V_1, V_2) = |\{\{u, v\} \in E \mid u \in V_1 \wedge v \in V_2\}| + \gamma(|V_1| - |V_2|)^2$

    γ: imbalance factor
- Comparison with Kernighan-Lin heuristic (Kernighan and Lin, 1970) for this problem:
  - SA better for random graphs
  - Kernighan-Lin better for geometric graphs

Average Cutsizes Found for Line, K-L, and Annealing (the Latter Two With and Without Good Starting Solutions), Compared to Best Cuts Ever Found[a]

	avg_deg $\approx n\pi d^2$				
$\lvert V\rvert$	5	10	20	40	Algorithm
500	4	26	178	412	(Best Found)
	38.1	89.4	328.4	627.8	Line
	36.1	89.7	221.0	436.3	K-L
	12.5	45.0	200.8	442.8	Line + K-L
	21.1	65.8	247.0	680.3	Anneal
	22.2	52.4	192.4	507.4	K-L + Anneal
	12.9	48.6	217.0	501.1	Line + Anneal
1000	3	39	222	737	(Best Found)
	47.2	123.7	381.8	1128.7	Line
	70.7	157.5	316.7	857.8	K-L
	15.7	64.6	272.0	825.5	Line + K-L
	41.2	120.4	355.3	963.8	Anneal
	38.1	99.9	295.0	833.8	K-L + Anneal
	15.8	54.7	262.5	839.6	Line + Anneal

[a]These figures do not take running time into account, and thus overestimate the relative efficacy of annealing.

Figure: Adapted from (Johnson et al., 1989, Table XI); graph class: random geometric graph $U_{n,d}$ on n points drawn uniformly from the *unit square*, expected average degree $\approx n\pi d^2$ (away from the boundary)

Enhancements of SA

- Nonmonotonic cooling schedule
  - *"Reheating"*
- Dynamic vs. static cooling schedule
- Deterministic investigation of the neighborhood
- Combination with other methods
- Parallelization

Conclusions for SA

- SA has been applied to several hundred applications
- SA is one of the most commonly used and also theoretically most studied metaheuristics
- Often it is simple to implement
- SA delivers good results for many problems but usually not excellent results compared to other leading methods
- SA is relatively time consuming

Tabu Search

Basic idea of tabu search (TS)

Glover (1986); Glover (1989); Hansen (1986): "steepest ascent/mildest descent", standard textbook: Glover and Laguna (1997)

- Based on a memory (history) of the previous optimization process, which is used to escape local optima, i.e., the tabu list
- Avoids cycles by forbidding the revisit of solutions in the memory
- Typically deterministically driven search
- Often best admissible neighbor is chosen:
  at each step the best *allowed* neighbor is accepted,
  even if this solution is worse than the current one

Tabu search – basic algorithm

procedure tabu search

begin
  $TL \leftarrow \emptyset$; // tabu list of length t_L
  $x \leftarrow$ initial solution;
  repeat:
    $X' \leftarrow$ non-tabu subset of $N(x)$ (w.r.t. TL);
    $x' \leftarrow$ best solution in X';
    add x' to TL;
    remove all elements from TL that are older than t_L iterations;
    $x \leftarrow x'$;
    if x is best solution found so far **then**
      store x as best solution;
  until stopping criteria satisfied;
end

Note: If X' is empty (all neighbors are tabu), one typically releases the oldest tabu attribute or reduces the tabu tenure.

Memory: the tabu list

- Store complete solutions explicitly
  - disadvantage: memory and time consuming
- Alternative: Store solution attributes (i.e., tabu attributes) of visited solutions
  $\rightarrow$ solutions are forbidden (tabu) if they contain tabu attributes
- Often, tabu attributes are represented as attributes of moves;
  the reverse moves stored in the tabu list are forbidden for the next t_L iterations
- Important parameter: tabu tenure (length of tabu list t_L)
- Often several tabu lists for different tabu attributes are used

Parameter: length of tabu list

- Choice of t_L is often very critical!
- Too short tabu lists would allow cycles
- Too long tabu lists would forbid many possible solutions and thus greatly restrict the search
- Appropriate length usually is problem specific; must be determined experimentally or
  - Choose randomly from an interval every few iterations
  - Lifespan of tabu attributes is set randomly for each move

Aspiration criteria

- Sometimes an unvisited solution could be forbidden (i.e., its attributes are in the tabu list) even if it is very good.
- Aspiration criteria override the tabu status of "interesting" solutions.
- Example of an often used aspiration criterion:
  - A forbidden solution is better than the best found solution.

Example: Tabu search for MAX-SAT

Hansen and Jaumard (1990); Selman et al. (1994)

Neighborhood structure: 1-flip neighborhood

Tabu attributes: modified variables

Tabu criterion: changing a variable is forbidden for t_L iterations

Aspiration criteria: if a move leads to a new best solution,
then override the tabu status.

Additional "tie-breaking" criterion: If several moves lead to equally good solutions, break the "tie" in favor of the move whose variable occurs in the largest number of unsatisfied clauses.

Example: TS for the graph coloring problem

Definition (graph coloring problem)

Given: graph $G = (V, E)$, number of colors k

Wanted: to each node $v \in V$ assign a color $x_v \in \{1, \ldots, k\}$, so that $x_u \neq x_v, \ \forall \{u, v\} \in E$

Optimization variants:

- Minimize the number of colors k for which such an assignment exists
- For *fixed* k: minimize the number of "violated" edges

Example: TS for the graph coloring problem (cont.)

Evaluation criterion: minimize number of "violated" edges

Neighborhood: change color of one node

Tabu attributes: pairs (v, j) with $v \in V$, $j \in \{1, \ldots, k\}$,
i.e., color assignment j to node v

Tabu criterion: if move $(v, j) \to (v, j')$ is applied, then attribute (v, j)
is forbidden for t_L iterations

Aspiration criterion: if move results in a new best solution,
then override tabu status

Restriction of the neighborhood: consider only nodes for color
assignments that are involved in a constraint violation

Attribute based hill climber (ABHC)

Whittley and Smith (2004)

- Originally developed for TSP and QAP
- Uses regional aspiration (vs. *global aspiration* for TS)
  - Value of an attribute is the objective value of the best found solution that contains the attribute (or infinite).
  - Solution is accepted if it is better than the value of at least one of its attributes
- Advantage: parameter free

Multi-level memory

Short term memory

- Essential part of TS, which was described previously

Medium term memory

- Intensification of the search
- "Restart" from *elite solutions*, e.g., with empty tabu list
- Solution attributes that are part of several elite solutions are fixed
- And several other strategies

Multi-level memory (cont.)

Long term memory

- Diversification of the search
- Turns focus on rarely explored regions of the search space
- Establishes solution attributes that are used rarely
  - e.g. by penalizing often- and rewarding rarely-used attributes

Other variants of memory

In general, the following variants of memory can be used and exploited through appropriate rules

recency-based: When was an attribute used the last time?

frequency-based: How often was an attribute used?

quality-based: What is the observed influence of an attribute on the objective function value?

influence-based: What is the influence of an attribute on the solution structure and feasibility?

Conclusions for TS

- Many different strategies for the
  - memory
  - diversification
  - intensification
- Many successful applications
- Often excellent results and relatively fast
- Usually, a relatively time-consuming fine-tuning is necessary

Configuration Checking

Configuration Checking (CC)

Cai et al. (2011)

- Principle is reminiscent of TS
- For a solution, after applying a move operator to some part of the solution, we keep track of the new local situation around that part, i.e., update a so-called configuration for all surrounding solution parts affected by this local change
- Applying a move operator to a part of the current solution is only allowed if the part's configuration has been altered since the last application of the move operator to that part (check by accessing the stored configuration)
- Mechanism to prevent cycling during local search
- Applied, e.g., to the problems Minimum Vertex Cover, MAX-SAT, SAT

Example: Solving SAT with CC

- Recall the SAT problem on a given set of clauses over the variables x_i.
- Consider the *neighborhood structure* *on variables* given by $\Gamma(x_i) = \{x_j \mid j \neq i,\ x_i \text{ and } x_j \text{ appear jointly in some clause}\}$, $\forall$variables x_i
- Example:

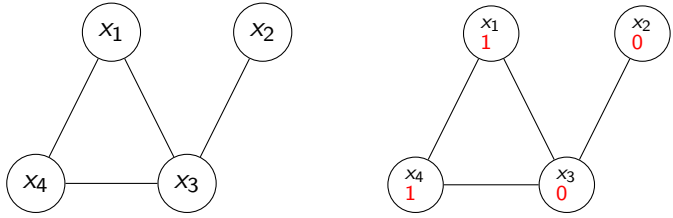
$$(x_1 \vee \overline{x}_3 \vee x_4) \wedge (\overline{x}_2 \vee x_3) \wedge (x_1) \wedge (x_1 \vee x_3) \wedge (\overline{x}_3 \vee \overline{x}_4)$$

with neighborhoods

$$\Gamma(x_1) = \{x_3, x_4\},\ \Gamma(x_2) = \{x_3\},\ \Gamma(x_3) = \{x_1, x_2, x_4\},\ \Gamma(x_4) = \{x_1, x_3\}.$$

Example: Solving SAT with CC

- Visualizing chosen neighborhood structure (left):



- For given truth value assignment $I : \{x_1, \ldots, x_4\} \to \{0, 1\}$, the configuration of x_i, $conf_{x_i}$, is defined as the restriction $I \restriction_{\Gamma(x_i)}$; realized e.g. as 0-1-valued dictionary of length $|\Gamma(x_i)|$.
- The assignment $(I(x_1), \ldots, I(x_4)) = (1, 0, 0, 1)$ is visualized above right.
  E.g., $conf_{x_1} = \{x_3 \mapsto 0, x_4 \mapsto 1\}$, and $conf_{x_3} = \{x_1 \mapsto 1, x_2 \mapsto 0, x_4 \mapsto 1\}$
- A simplified "history of changes" of configurations will be used to forbid certain local search directions (to be explained soon)

Typical framework for SAT via local search

Given: Variables x_i, clauses C_j

procedure Local Search for SAT $(\{x_1, \ldots, x_n\}, \{C_1, \ldots, C_m\})$

begin
  $x \leftarrow$ initial truth value assignment for x_i, $i = 1, \ldots, n$;
  repeat:
    if x is a satisfying assignment **then return** x **end**
    Pick variable $z \in \{x_1, \ldots, x_n\}$; `// by a heuristic`
    $x \leftarrow x$ with z flipped;
  until stopping criteria satisfied;
  return no solution found
end

Incorporating CC into local search: Naive approach

procedure Local Search for SAT $(\{x_1, \ldots, x_n\}, \{C_1, \ldots, C_m\})$

begin
  $x \leftarrow$ initial truth value assignment for x_i, $i = 1, \ldots, n$;
  $conf_{x_i} \leftarrow \text{undefined}$, $i = 1, \ldots, n$;
  repeat:
    if x is a satisfying assignment **then return** x **end**
    Pick variable $z \in \{x_1, \ldots, x_n\}$; `// by a heuristic`
    if $conf_z \neq \text{undefined}$ **then**
      if restriction $x \restriction_{\Gamma(z)} = conf_z$ **then continue end**
    end
    $x \leftarrow x$ with z flipped;
    $conf_z \leftarrow x \restriction_{\Gamma(z)}$
  until stopping criteria satisfied;
  return no solution found
end

Incorporating CC into local search: Practical approach

Practicality at the price of a loss in accuracy.

procedure Local Search for SAT ($\{x_1, \ldots, x_n\}$, $\{C_1, \ldots, C_m\}$)

begin
  $x \leftarrow$ initial truth value assignment for x_i, $i = 1, \ldots, n$;
  Initialize array of Booleans $configCh_{x_i} \leftarrow 1$, $i = 1, \ldots, n$;
  repeat:
    if x is a satisfying assignment **then return** x **end**
    Pick variable $z \in \{x_1, \ldots, x_n\}$; `// by a heuristic`
    if $configCh_z = 1$ **then**
      $x \leftarrow x$ with z flipped;
      $configCh_z \leftarrow 0$
      for $y \in \Gamma(z)$ **do** $configCh_y \leftarrow 1$ **end**
    end
  until stopping criteria satisfied;
  return no solution found
end

Further Comments on CC (SAT example)

- Keeping/updating configuration per flip might be expensive
- Remedy: use reduced information on change of configuration status:
  - Track if the configuration has altered since last move operator application, ignoring the possibility that an earlier configuration is revisited
  - (Storage of just one bit of information needed)
- Note that the method is parameter free (nothing comparable to a tabu tenure)
- In practice, the candidate set is restricted to the variables with $\mathit{configCh} = 1$ (with a random-walk fallback if none exists), since otherwise iterations may pass without any flip
- Further applications: Optimal Golomb Ruler Problem ($\rightsquigarrow$ later), Graph Coloring Problem

Comparison: Tabu Search vs Configuration Checking

- TS introduced in 1986 vs. CC introduced in 2011
- Empirical comparison of TS vs CC in the context of SAT via a clause-weighting scheme with smoothing (SWT) is carried out in Cai and Su (2011).
- For other problems (e.g., Optimal Golomb Ruler) it turns out that combining TS with CC provides a decisive improvement of success rate (Polash et al., 2017)

Local search for SAT: Tabu Search vs. CC

Instance Class	#inst.	SW_{cc} suc	SW_{cc} avg time	SW_{cc} avg step	opt tt	SW_{tabu} suc	SW_{tabu} avg time	SW_{tabu} avg step	time ratio
3SAT-2000vars	10	100%	1.3s	2,546,510	6	100%	1.3s	**2,132,847**	1
3SAT-4000vars	10	100%	21.6s	30,884,521	12	100%	**19.6s**	**27,143,316**	1.102
3SAT-6000vars	10	100%	43s	54,544,473	13	100%	**35s**	**43,825,439**	1.228
3SAT-8000vars	10	**99.1%**	**56s**	**62,157,131**	15	93.3%	85s	85,115,113	0.658
3SAT-10000vars	10	**100%**	**29s**	**28,706,690**	18	100%	106s	93,927,502	0.273
3SAT-12000vars	10	**100%**	**93s**	**76,706,252**	20	90.7%	339s	355,712,224	0.274
3SAT-14000vars	10	**100%**	**109s**	**80,697,100**	25	85%	522s	450,179,222	0.208
3SAT-16000vars	10	**100%**	**62s**	**40,739,558**	24	84.9%	462s	407,261,877	0.134
3SAT-18000vars	10	**100%**	**81s**	**48,248,766**	28	72.3%	626s	509,130,721	0.129
3SAT-20000vars	10	**92%**	**326s**	**187,625,156**	30	4.3%	962s	628,926,901	0.339
3SAT-22000vars	10	**99.3%**	**207s**	**103,979,675**	38	5.9%	973s	640,817,872	0.212
3SAT-24000vars	10	**93.5%**	**215s**	**102,029,511**	30	4.1%	989s	639,887,021	0.217
3SAT-26000vars	10	**97.5%**	**195s**	**87,829,798**	36	5.2%	973s	595,811,110	0.204
All instances	130	**98.6%**	**110s**	**69,745,780**	n/a	65.1%	468s	344,605,474	0.235

Performance results of SW_{cc} and SW_{tabu} (with optimal tabu tenure) on large random 3-SAT instances. For each row, the time ratio is the average time of SW_{cc} divided by the average time of SW_{tabu}.

Figure: Adapted from Cai and Su (2011)

Golomb Ruler illustration

Goal: For a given number m of marks, find integer positions $0 = a_1 < a_2 < \cdots < a_m$ such that all pairwise distances $a_j - a_i$ are distinct and the length a_m is minimal.

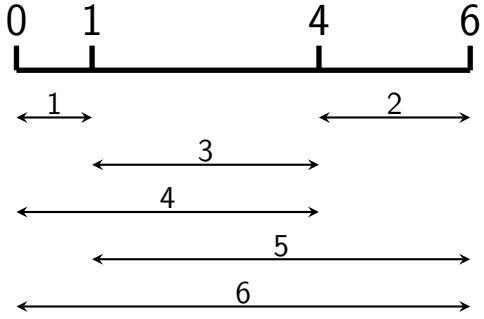
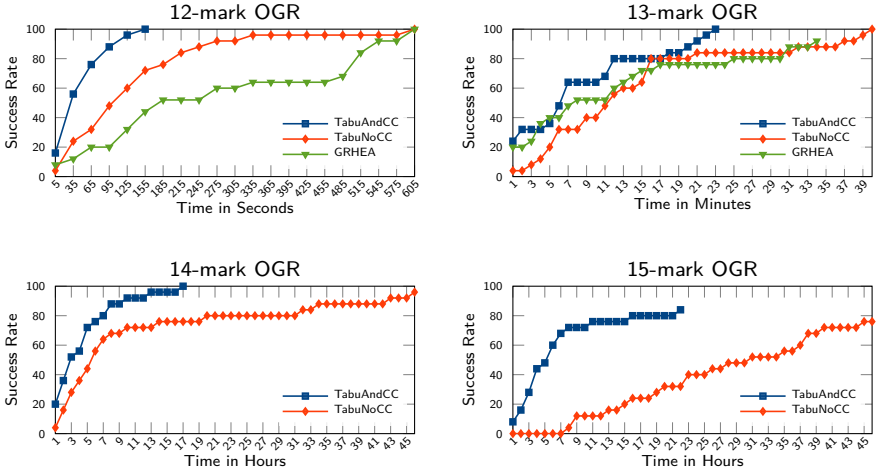


Figure: Optimal Golomb ruler with $m = 4$ marks (length 6). Adapted from

https://upload.wikimedia.org/wikipedia/commons/0/05/Golomb_Ruler-4.svg

Comparison: Tabu Search vs. Tabu Search+CC



TabuAndCC for a 16-mark ruler													
Time in Hours	1	3	5	7	9	11	13	15	17	19	21	23	25
Success Rate (%)	4	8	16	20	24	24	24	32	40	52	52	56	60

Figure: Optimal Golomb Ruler Problem. Adapted from Polash et al. (2017).

Bibliography I

E. Bonomi and J.-L. Lutton. The *N*-city travelling salesman problem: Statistical mechanics and the metropolis algorithm. *SIAM Review*, 26(4):551–568, 1984.

S. Cai and K. Su. Local search with configuration checking for SAT. In *2011 IEEE 23rd International Conference on Tools with Artificial Intelligence*, pages 59–66. IEEE, 2011.

S. Cai, K. Su, and A. Sattar. Local search with edge weighting and configuration checking heuristics for minimum vertex cover. *Artificial Intelligence*, 175(9-10):1672–1696, 2011.

F. Glover. Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research*, 13(5):533–549, 1986.

F. Glover. Tabu search—part I. *ORSA Journal on Computing*, 1(3):190–206, 1989.

F. Glover and M. Laguna. *Tabu search*. Kluwer Academic Publishers, Boston, MA, 1997.

P. Hansen. The steepest ascent mildest descent heuristic for combinatorial optimization. In *Congress on Numerical Methods in Combinatorial Optimization, Capri, Italy*, 1986.

P. Hansen and B. Jaumard. Algorithms for the maximum satisfiability problem. *Computing*, 44(4): 279–303, 1990.

D. S. Johnson and L. A. McGeoch. The traveling salesman problem: A case study in local optimization. In E. H. L. Aarts and J. K. Lenstra, editors, *Local Search in Combinatorial Optimization*, pages 215–310. Wiley, Chichester, UK, 1997.

D. S. Johnson, C. R. Aragon, L. A. McGeoch, and C. Schevon. Optimization by simulated annealing: An experimental evaluation; part I, graph partitioning. *Operations Research*, 37(6):865–892, 1989.

Bibliography II

B. W. Kernighan and S. Lin. An efficient heuristic procedure for partitioning graphs. *The Bell System Technical Journal*, 49(2):291–307, 1970.

S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220 (4598):671–680, 1983.

M. M. A. Polash, M. A. H. Newton, and A. Sattar. Constraint-based search for optimal Golomb rulers. *Journal of Heuristics*, 23:501–532, 2017.

B. Selman, H. A. Kautz, and B. Cohen. Noise strategies for improving local search. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI-94)*, pages 337–343, 1994.

I. M. Whittley and G. D. Smith. The attribute based hill climber. *Journal of Mathematical Modelling and Algorithms*, 3:167–178, 2004.